Homework 5 — The for loop

Due Sunday 18 October, 23:59, on Canvas. The deadline is fixed; late work is not received.

Individual work. One file, two functions and a `main()`. The week-3 rule continues with a twist: **every `for` comes with one sentence saying how many times it runs** — written as a number or a formula in `n`. (For a `for` over a `range`, that sentence *is* the termination proof.)

What you submit

One file, `A_Matricula_hw5.py`, in the exam format (name comment on line 1, algorithms in `"""`), containing:

1. **`is_perfect(n)`** — returns `True` when `n` equals the sum of its proper divisors (every positive divisor below `n`), `False` otherwise. `6 = 1 + 2 + 3` is perfect; `28 = 1 + 2 + 4 + 7 + 14` is perfect; 12 is not (`1 + 2 + 3 + 4 + 6 = 16`). A `for` over `range(1, n)` with an `if`. Numbers below 2 are not perfect. No `input()`, no `print()` inside.
2. **`longest_collatz(limit)`** — returns the number from 1 to `limit` (inclusive) whose Collatz sequence takes the **most** steps to reach 1. Reuse **your own `collatz_steps(n)` from Homework 4** — paste it into this file — and put a `for` around it with a *best-so-far*: remember the number with the most steps seen, replace it only when beaten. If two numbers tie, keep the smaller one (that is what *replace only when beaten* already does — say why in the algorithm).
3. **`main()`** — asks which to run (1 or 2), reads the number, refuses values below 1 with `Input out of range.`, otherwise calls the function and prints the result. Last line calls `main()`.

Plus, as a photo or scan: **the two algorithms** on paper, each `FOR` with its *how many times* sentence, and **your shell tests** — what you typed and what came back.

Shell tests — required

```
is_perfect(6)           True
is_perfect(28)          True
is_perfect(12)          False
is_perfect(1)           False
is_perfect(496)         True
longest_collatz(1)      1
longest_collatz(10)     9       (19 steps)
longest_collatz(100)    97      (118 steps)
```

`longest_collatz(100)` calls `collatz_steps` a hundred times and takes well under a second. Try `longest_collatz(10000)` and write down how long it felt — Friday's lesson about loops inside loops, measured.

Rubric — published

Both functions return the right values on all eight shell tests (no `print`, no `input` inside)	40 %
Algorithms on paper, every `FOR` with its range and its *how many times* sentence; the best-so-far explained, including the tie	30 %
`main()` reads, checks the range with the exact message, calls and prints; file ends with `main()`; `collatz_steps` is present and is the one from HW4	20 %
Shell tests written down as run	10 %

Two points off for each missing technical requirement (name comment, algorithms in `"""`, file name, meaningful names).

If you want more

`longest_collatz` returns one number. Return **two** — the number and its step count: `return best_n, best_steps`. Then plot nothing yet (week 8), but print a bar of `*` per 10 steps for every `n` up to 30 — a picture of the Collatz landscape made of Friday's `bars`.